

Reviewer search checklist

DEVICE SOFTWARE, CYBERSECURITY AND INTERFACES

Use the topic checks while searching the submission. Each checkbox identifies a specific check; Where and Search guide navigation, Why gives its purpose, and Details to check lists the information to look for.

Tick after examining the applicable check. A tick means examined, with acceptability judged separately. Use existing review notes for findings.

☐ U01 Confirm the evidence identity

Where: Document cover; headers; configuration tables • **Search:** revision, build, configuration

Why: Keeps evidence tied to the correct object.

Details to check: Document/revision and original page/table; software/build; relevant time, population and configuration; distinguish printed and PDF page numbers.

☐ U02 Establish which expectation applies

Where: Review scope; applicable guidance/standard; protocol • **Search:** scope, level, class, criterion

Why: A checklist prompt alone does not make information mandatory.

Details to check: Relevant device/function; documentation route; exact edition/section; requirement versus recommendation or sponsor commitment; justified alternative or nonapplicability.

☐ U03 Read the original content around search hits

Where: Body text; tables; footnotes; figures; appendices • **Search:** synonyms, appendix, referenced ID

Why: Keywords and document titles are only navigation clues.

Details to check: Actual content; units; shared headers; row exceptions; referenced evidence; unreadable/unexamined scope. A keyword miss does not prove absence.

☐ U04 Separate what is claimed from what is demonstrated

Where: Plan; execution record; result; linked verification • **Search:** planned, completed, actual, pass

Why: A statement of completion may omit the underlying observation.

Details to check: Plan versus result; expected versus actual; reported outcome; conditions; accessible supporting evidence. Supplied information can document a failed result.

☐ U05 Check conflicting and adverse evidence

Where: Detailed results; deviations; changes; open-item lists • **Search:** failed, deferred, excluded, superseded

Why: A favorable conclusion may conceal a material qualification.

Details to check: Compatible IDs/versions/populations; failures and exceptions; contradictory values; governing evidence or unresolved conflict; basis for any bounded absence observation.

6.0.0 / search checklist 2.0 · 2026-10-06

Topic finder — T00 to T15

Select a topic to jump to its checks. Page numbers refer to the full PDF; Word pagination may differ.

Topic	What to search for	PDF pages
T00	Submission/module context and scope	4
T01	Documentation level and software safety classification	5
T02	Software purpose, functions, inputs and outputs	6
T03	Software versions and configurations	7
T04	Safety risk management	8–9
T05	Software requirements	10
T06	System and software architecture	11
T07	Software design	12
T08	Development, configuration management and maintenance practices	13–14
T09	Software verification and validation	15–16
T10	Version history and changes	17
T11	Unresolved software anomalies	18
T12	Third-party / SOUP / off-the-shelf software	19
T13	Cybersecurity risk management process and report	20
T14	Threat modeling	21
T15	Cybersecurity risk assessment	22

For clinical/analytical performance, manufacturing, biocompatibility or other domains, use the appropriate review resources. These 31 topics cover software and its interfaces.

Topic finder — T16 to T30

Select a topic to jump to its checks. Page numbers refer to the full PDF; Word pagination may differ.

Topic	What to search for	PDF pages
T16	Software bill of materials	23
T17	Vulnerabilities and component support	24
T18	Security implications of unresolved anomalies	25
T19	Cybersecurity traceability	26
T20	Cybersecurity measures and metrics	27
T21	Security controls and requirements	28–29
T22	Security architecture views	30
T23	Cybersecurity testing	31–32
T24	Cybersecurity labeling	33
T25	Postmarket cybersecurity management	34
T26	Cyber-device applicability statements	35
T27	Usability and human factors	36–37
T28	Standards and conformity claims	38
T29	Software/device labeling	39
T30	Machine-learning functions and controls	40–41

For clinical/analytical performance, manufacturing, biocompatibility or other domains, use the appropriate review resources. These 31 topics cover software and its interfaces.

T00 — Submission/module context and scope

Applies: Start here for every review; use modular-submission checks only when that route applies.

☐ T00.01 Identify the device and software functions under review

Where: Cover letter; executive summary; device description • **Search:** applicant, device, module, intended use

Why: Sets the boundary for every later check.

Details to check: Applicant and device identity; software function or module; intended use; pathway; included and excluded functions.

☐ T00.02 Find the governing submission snapshot

Where: Submission index; document register; amendment correspondence • **Search:** revision, amendment, supersedes, snapshot

Why: Prevents combining superseded and current evidence.

Details to check: Document titles and revisions; submitted dates; amendments; stated supersession; software/configuration addressed by each change.

☐ T00.03 Check that referenced evidence is accessible

Where: Index; report cross-references; appendices and attachments • **Search:** appendix, attachment, report ID, see also

Why: A listed document may supply no examinable content.

Details to check: Referenced title/ID and revision; actual attachment; relevant pages, tables and images; missing or unreadable regions.

☐ T00.04 Identify earlier evidence being relied upon

Where: Prior-submission references; bridging rationale; review correspondence • **Search:** prior submission, previously reviewed, unchanged

Why: Earlier evidence may address a different device or scope.

Details to check: Prior reference and decision scope; product/version; reused evidence; intervening changes; rationale linking it to this submission.

☐ T00.05 Locate unresolved commitments and later changes

Where: Module correspondence; response letters; updated plans • **Search:** pending, will provide, follow-up, post-acceptance

Why: Distinguishes completed evidence from future work.

Details to check: Agreed deliverable and scope; target event/date; current status; submitted response; changed evidence or conditions after module acceptance.

☐ T00.06 Route material beyond software review

Where: Contents; device description; performance and manufacturing sections • **Search:** clinical, analytical, manufacturing, electrical safety

Why: Software coverage does not settle other device questions.

Details to check: Relevant source and affected function; relationship to software evidence; appropriate review domain; any explicitly unexamined scope.

Related checks: T03, T09, T10, T28 | Source pointers: S1 §III; S14 §VI

T01 — Documentation level and software safety classification

Applies: Check the claimed documentation level and any claimed software safety class as separate decisions.

□ T01.01 Find the documentation-level decision and rationale

Where: Documentation-level evaluation; risk file • **Search:** Basic, Enhanced, documentation level

Why: Determines the applicable software-documentation route.

Details to check: Selected level; assessed device/functions; supporting failure and harm scenarios; cited guidance basis; rationale for the chosen route.

□ T01.02 Check the harm assumptions behind the level

Where: Hazard analysis; classification rationale • **Search:** before controls, serious injury, death, failure

Why: A category label alone does not explain the decision.

Details to check: Relevant failure modes; possible harm; conditions considered; treatment of risk controls; support for severity assumptions.

□ T01.03 Find the software safety-class decision, if claimed

Where: Safety-classification record; conformity documentation • **Search:** class A, class B, class C, 62304

Why: This classification scheme has its own scope and criteria.

Details to check: Exact standard/edition; system or item classified; selected class; rationale; contributing software failures; external risk controls relied upon.

□ T01.04 Check separately classified software items

Where: Architecture; segregation rationale; class assignment • **Search:** segregation, software item, external control

Why: A lower item class depends on the stated boundary.

Details to check: Item boundaries; interfaces and dependencies; segregation mechanism; relied-on hardware or external controls; supporting evidence.

□ T01.05 Compare classification with the actual device scope

Where: Device description; safety and security risk files • **Search:** intended use, configuration, cybersecurity harm

Why: Changes or omitted functions can undermine the rationale.

Details to check: Same intended use and release; included functions; security-related harm; later changes; unresolved inconsistencies between classifications and risks.

Related checks: T04, T06, T13, T28 | Source pointers: S1 §§V, VI.A; S10 §4.3

T02 — Software purpose, functions, inputs and outputs

Applies: Apply to the software functions under review and to other functions that can affect them.

□ T02.01 Identify intended use, users and workflow

Where: Device description; intended-use statement; IFU • **Search:** intended user, population, workflow, automated

Why: Gives requirements and tests their operating context.

Details to check: Clinical purpose; intended users and population; environment; user actions; automated steps; software role in the device.

□ T02.02 Check each material input and its limits

Where: Interface specification; SRS; data dictionary • **Search:** input, format, units, range, validation

Why: Input assumptions define meaningful test boundaries.

Details to check: Input source; data type/format; units; allowed and validated ranges; quality checks; handling of missing, invalid or out-of-range data.

□ T02.03 Check each material output and its recipient

Where: Output specification; report examples; IFU • **Search:** output, result, display, recipient, units

Why: An output can be technically correct yet misinterpreted downstream.

Details to check: Meaning and units; format; receiving user/system; interpretation limits; error or invalid-result indications; links to underlying functions.

□ T02.04 Identify algorithms and automated decisions

Where: Software description; algorithm overview; workflow diagrams • **Search:** algorithm, calculation, decision, preprocessing

Why: Locates functions needing requirements and performance support.

Details to check: Method and purpose; processing sequence; assumptions; configurable settings; decision thresholds where used; relevant limitations.

□ T02.05 Find supported platforms and deployment conditions

Where: System requirements; installation guide; architecture • **Search:** hardware, operating system, hosting, cloud

Why: A platform family may conceal untested configurations.

Details to check: Hardware and OS versions; local/cloud deployment; required resources; dependencies; permitted configurations; tested versus supported limits.

□ T02.06 Check external interfaces and other-function interactions

Where: Interface-control documents; system diagrams; interoperability analysis • **Search:** protocol, counterpart, other function, data flow

Why: Shared resources and interfaces can affect reviewed functions.

Details to check: Source/target; protocol/version; data direction; counterpart products; timing/error behavior; shared resources; safety/effectiveness impact.

Related checks: T05, T06, T09, T27, T29 | Source pointers: S1 §VI.B; S15 §§V–VI

T03 — Software versions and configurations

Applies: Apply to each proposed, tested, released or historical baseline used in the evidence.

□ T03.01 Identify the proposed release

Where: Release manifest; version history; submission description · **Search:** proposed version, release, build ID, hash

Why: Establishes the software object the review concerns.

Details to check: Version and naming scheme; build ID/hash if used; build/release date; device/module identity; stated scope of a version-only identifier.

□ T03.02 Identify the versions actually tested

Where: Test-report headers; execution logs; configuration tables · **Search:** tested build, baseline, configuration

Why: Results support the tested configuration first.

Details to check: Software/build; hardware and OS; component versions; configuration settings; model/data version if relevant; report-to-run consistency.

□ T03.03 Find the controlled configuration baseline

Where: Configuration register; build manifest; toolchain record · **Search:** configuration item, compiler, dependency, baseline

Why: A shared product version may hide different contents.

Details to check: Configuration items and versions; build tools/settings where material; dependency baseline; reproducibility information; approved variants.

□ T03.04 Check user-visible version identification

Where: About screen; labels; IFU; service instructions · **Search:** version display, About, software identifier

Why: Users and service staff need to identify the relevant release.

Details to check: Displayed software version; access method; relationship to build identity; device identifiers kept distinct from software version.

□ T03.05 Check changes between tested and proposed versions

Where: Change log; impact assessment; regression report · **Search:** change since, freeze, bridge, equivalence

Why: A different build needs a supported relevance argument.

Details to check: Exact differences; affected functions and dependencies; risk impact; bridging rationale; repeated tests; dates as context, not proof of content identity.

Related checks: T09, T10, T16, T23, T29 | Source pointers: S1 §§VI.H–I; S10 §§5.8, 8.1–8.3

T04 — Safety risk management (1/2)

Applies: Apply to software-related risks within the device risk-management scope.

☐ T04.01 Find the risk plan and decision criteria

Where: Risk-management plan; approved risk procedure • **Search:** acceptability, responsibility, overall residual

Why: Risk judgments need an established method and scope.

Details to check: Device/lifecycle scope; responsibilities; approval; risk-acceptability criteria; overall-risk method; verification and post-production arrangements.

☐ T04.02 Follow failure scenarios through to harm

Where: Hazard analysis; use-related risk analysis; software risk analysis • **Search:** hazard, hazardous situation, sequence, harm

Why: A failure label alone does not describe patient or user risk.

Details to check: Intended use/misuse; causes and event sequence; hazardous situation; resulting harm; software contribution; initial severity/probability and their basis.

☐ T04.03 Check the selected risk controls

Where: Risk-control table; requirements; design records • **Search:** control option, inherent safety, protective measure

Why: The control should address the stated risk mechanism.

Details to check: Control ID and rationale; affected hazard; implementation requirement/design; alternative options considered; reliance on warnings or external actors.

☐ T04.04 Find evidence that each relied-on control is implemented

Where: Trace matrix; design verification; test records • **Search:** implemented, control verification, requirement ID

Why: A proposed control may never have reached the tested release.

Details to check: Control-to-requirement/design link; implemented version; verification method; executed result; unresolved implementation failures.

☐ T04.05 Find evidence that each relied-on control is effective

Where: Effectiveness tests; validation; human-factors evidence • **Search:** effectiveness, residual risk, user task

Why: Existence of a control does not establish risk reduction.

Details to check: Risk-reduction objective; conditions and criterion; actual result; intended users/environment for human controls; remaining limitations.

☐ T04.06 Check risks introduced or changed by controls

Where: Updated hazard analysis; change-impact review • **Search:** new risk, increased risk, side effect

Why: A control can create another hazardous situation.

Details to check: New hazards; changed likelihood/severity; affected functions; further controls; associated verification and residual-risk decisions.

Related checks: T05, T09, T11, T15, T27, T29 | Source pointers: S1 §VI.C; S11 §§4.4–10; S10 §7

T04 — Safety risk management (2/2)

Applies: Apply to software-related risks within the device risk-management scope.

☐ **T04.07 Check individual residual-risk decisions**

Where: Risk evaluation; benefit-risk analysis where invoked · **Search:** residual, unacceptable, benefit-risk

Why: Individual risk acceptance needs its own supporting argument.

Details to check: Post-control risk and criterion; supporting evidence; justification where benefits are invoked; unresolved anomalies; necessary disclosures.

☐ **T04.08 Find the overall risk conclusion and ongoing review**

Where: Risk-management report; production/post-production plan · **Search:** overall residual, release review, monitoring

Why: Separate acceptable rows do not by themselves establish overall acceptability.

Details to check: Combined residual-risk rationale; completed controls; review/approval; information sources; reassessment triggers; user disclosures; follow-up responsibilities.

Related checks: T05, T09, T11, T15, T27, T29 | Source pointers: S1 §VI.C; S11 §§4.4–10; S10 §7

T05 — Software requirements

Applies: Apply to the current requirements for the functions and configurations being reviewed.

□ T05.01 Identify the governing requirements baseline

Where: SRS cover/revision history; requirements export • **Search:** SRS, approved, baseline, revision

Why: Trace links are meaningful only for the relevant requirements.

Details to check: Document set and revision; approval; system/item scope; export date; changed or superseded requirements; consistent IDs and namespaces.

□ T05.02 Check that functional requirements are specific

Where: SRS; user-needs allocation; interface requirements • **Search:** shall, input, output, behavior, condition

Why: Vague behavior cannot support a clear verification comparison.

Details to check: Required behavior; triggering conditions; input/output; units and limits; boundary cases; error handling; applicable configurations.

□ T05.03 Find measurable performance and constraint criteria

Where: SRS performance sections; timing/resource specifications • **Search:** latency, accuracy, capacity, limit, acceptance

Why: A test pass needs a defined target and conditions.

Details to check: Threshold/range; units; load and operating conditions; resource constraints; rationale or higher-level need supporting material limits.

□ T05.04 Locate safety and security requirements

Where: Risk-control allocation; security requirements; SRS • **Search:** risk control, safety requirement, security requirement

Why: Risk controls can disappear between analysis and implementation.

Details to check: Requirement ID and text; linked hazard/threat/control; safety/security significance; responsible software item; associated verification method.

□ T05.05 Follow requirements into design and executed verification

Where: Trace matrix; SDS; test cases and results • **Search:** traceability, implements, verifies, test ID

Why: A case-definition link does not establish an executed outcome.

Details to check: Requirement/design/case/run IDs; compatible versions; actual outcome; uncovered requirements; design or tests lacking a justified requirement link.

□ T05.06 Check revised requirements against earlier testing

Where: Change records; SRS history; regression analysis • **Search:** revised requirement, impact, retest

Why: An older result may not address a changed target.

Details to check: Changed wording/criterion; effective version/date; affected controls and tests; rationale for reuse or retest; resulting evidence.

Related checks: T04, T07, T09, T19, T21 | Source pointers: S1 §VI.D; S10 §5.2

T06 — System and software architecture

Applies: Apply to the system and software boundaries supporting the reviewed functions.

□ T06.01 Identify the system elements and allocated functions

Where: Architecture diagram; architecture narrative • **Search:** module, item, layer, allocation

Why: Shows where reviewed behavior is implemented.

Details to check: Named elements; responsibilities; hardware/software allocation; third-party components; correspondence with the software description.

□ T06.02 Trace internal interfaces

Where: Interface-control documents; data-flow diagrams • **Search:** source, target, interface, data type

Why: Hidden connections can invalidate isolation or test coverage.

Details to check: Connected elements; direction; data/control flow; format and timing; error paths; consistency between diagrams and explanatory text.

□ T06.03 Locate external systems and deployment boundaries

Where: Deployment diagram; network architecture; service description • **Search:** cloud, hospital, external, hosted, service

Why: The operating system can extend beyond the device enclosure.

Details to check: External services and counterpart products; ownership; interface purpose; deployment configuration; included and excluded systems.

□ T06.04 Check claimed segregation or isolation

Where: Segregation rationale; architecture verification • **Search:** isolation, partition, separation, independence

Why: A diagram boundary alone does not demonstrate separation.

Details to check: Mechanism enforcing separation; shared resources; failure propagation; dependencies; supporting verification; links to classification and risk controls.

□ T06.05 Reconcile architecture with requirements and the release

Where: Configuration manifest; SRS; alternate architecture views • **Search:** component name, version, optional, other function

Why: Different names or versions can hide a scope mismatch.

Details to check: Element identity across views; proposed configuration; optional/other-function interactions; omitted elements; change impact and unresolved inconsistencies.

Related checks: T02, T04, T07, T14, T22 | Source pointers: S1 §VI.E and Appendix B; S10 §5.3

T07 — Software design

Applies: Use the established documentation route: detailed design is not a routine Basic-level submission request; examine it when supplied, applicable or needed for a specific question.

□ T07.01 Locate the design for each material function

Where: SDS; algorithm specification; detailed-design annex • **Search:** design element, implements, algorithm

Why: Connects required behavior to the implementation approach.

Details to check: Design element ID; software item; implemented requirement; logic/algorithm; assumptions; configuration scope.

□ T07.02 Check interface and data-handling design

Where: SDS interface sections; data structures; state diagrams • **Search:** state, transition, data structure, timeout

Why: High-level architecture may omit decisive failure behavior.

Details to check: Input/output structures; interface behavior; state transitions; error/timeout handling; exceptional conditions; safety/security implications.

□ T07.03 Check that design links cover the relevant requirements

Where: Requirement-to-design matrix; design review • **Search:** trace, unimplemented, orphan, allocation

Why: A design description may omit or add behavior.

Details to check: Requirement and design IDs; compatible revisions; uncovered requirements; untraced design elements; rationale for introduced behavior.

□ T07.04 Find design-verification evidence

Where: Design reviews; detailed-design verification records • **Search:** design verification, review finding, resolution

Why: A design statement alone does not show it was checked.

Details to check: Reviewed revision; method and criteria; findings; corrections; unresolved items; relationship to implementation and testing.

□ T07.05 Check design changes against the tested baseline

Where: SDS history; change-impact records; test configuration • **Search:** revision, approval, design change

Why: Later design content may not be represented by earlier tests.

Details to check: Initial and revised design scope; affected items/interfaces; governing baseline; impact assessment; relevant verification or retesting.

Related checks: T05, T06, T09, T21 | Source pointers: S1 §VI.F; S10 §5.4; S13 §5.4

T08 — Development, configuration management and maintenance practices (1/2)

Applies: Follow the applicable documentation level and conformity route. Internal process records are not automatic submission deliverables.

□ T08.01 Find the claimed lifecycle and documentation route

Where: Development plan; process description; conformity declaration • **Search:** lifecycle, process, 62304, declaration

Why: Determines which process evidence the sponsor relies upon.

Details to check: Lifecycle activities; responsibilities; deliverables; claimed clauses/edition; submitted procedures or descriptions; justified tailoring.

□ T08.02 Check configuration and change control

Where: Configuration-management plan; change procedure • **Search:** baseline, change control, status accounting

Why: Uncontrolled changes undermine evidence identity.

Details to check: Configuration identification; approval of changes; version/status tracking; source custody; release baseline; links to impact assessment and verification.

□ T08.03 Check traceability practices

Where: Development plan; traceability procedure; tool exports • **Search:** traceability, user need, requirement, test

Why: Process links should support product-specific evidence chains.

Details to check: Objects linked; forward/reverse scope; maintenance responsibility; change handling; tool/export configuration; actual release-specific application where relevant.

□ T08.04 Find coding and development-tool controls

Where: Coding standards; tool register; build procedure • **Search:** coding standard, compiler, static analysis, tool version

Why: Development settings can affect repeatability and defects.

Details to check: Applicable coding rules; tools/versions/settings; review or verification approach; deviations; external-component configuration controls.

□ T08.05 Check maintenance and regression arrangements

Where: Maintenance plan; regression policy; update process • **Search:** maintenance, impact analysis, regression, update

Why: The release needs a controlled path for future changes.

Details to check: Change/problem intake; impact analysis; regression selection; update verification; release approvals; supplier changes; responsibilities and commitments.

Related checks: T03, T09, T10, T12, T25 | Source pointers: S1 §VI.G; S10 §§4.4, 5.1, 6, 8–9; S13 §§4–6, 8–9

T08 — Development, configuration management and maintenance practices (2/2)

Applies: Follow the applicable documentation level and conformity route. Internal process records are not automatic submission deliverables.

☐ **T08.06 Check problem resolution through closure**

Where: Problem-report procedure; linked change/test records · **Search:** problem report, investigate, resolution, trend

Why: Closing a ticket does not demonstrate resolution of its effects.

Details to check: Investigation; cause and impact; affected products/versions; corrective change; verification; communication; retained unresolved items.

☐ **T08.07 Check legacy-software justification when invoked**

Where: Legacy assessment; gap analysis; remediation plan · **Search:** legacy, gap, continued use, remediation

Why: A legacy label does not establish adequate supporting evidence.

Details to check: Legacy boundary; risk evaluation; available evidence; process/evidence gaps; additional work; closure status; rationale for continued use.

Related checks: T03, T09, T10, T12, T25 | Source pointers: S1 §VI.G; S10 §§4.4, 5.1, 6, 8–9; S13 §§4–6, 8–9

T09 — Software verification and validation (1/2)

Applies: Apply the established documentation route. S1 §VI.H covers system protocols/results at Basic level and additionally unit/integration protocols/reports at Enhanced level; use targeted detail for the review question.

□ T09.01 Find the verification and validation objectives

Where: V&V plan; validation protocol; requirements matrix • **Search:** objective, user need, unit, integration, system

Why: Establishes what each activity was intended to demonstrate.

Details to check: Verification versus validation purpose; levels; user needs/requirements/controls covered; planned methods; scope exclusions and rationale.

□ T09.02 Find the governing protocol and acceptance criteria

Where: Approved protocol; test-case definitions; revision history • **Search:** expected result, criterion, approved, revision

Why: An outcome needs a defined target and governing method.

Details to check: Case ID; protocol revision; criterion and units; operating conditions; approval/execution dates; deviations or amended criteria.

□ T09.03 Match the tested configuration to the proposed release

Where: Test headers; execution logs; release manifest • **Search:** tested version, build, baseline, configuration

Why: Evidence from a different configuration needs a bridge.

Details to check: Software/build; platform/dependencies; tools and data; settings; proposed release; intervening differences; equivalence or retest evidence.

□ T09.04 Find actual observations behind reported outcomes

Where: Result tables; logs; appendices; original reports • **Search:** actual result, observed, measured, pass, fail

Why: A Pass label may not show what was observed.

Details to check: Case/run ID; actual value or behavior; units; criterion; reported outcome; source row/log; qualifications beside the result.

□ T09.05 Check test conditions and boundary coverage

Where: Methods; test-data description; table notes • **Search:** boundary, load, logging disabled, worst case

Why: A favorable result may depend on restricted conditions.

Details to check: Input ranges; load/configuration; relevant boundary/error conditions; footnotes; exclusions; justification for representativeness.

□ T09.06 Reconcile execution counts and coverage

Where: Execution register; result tables; coverage report • **Search:** planned, executed, blocked, deferred, retest

Why: Overlap and different counting units can mislead.

Details to check: Cases versus runs; source population; executed/pass/fail/blocked/deferred totals; repeated runs; overlap; unexplained differences.

Related checks: T03, T04, T05, T10, T11, T23 | Source pointers: S1 §VI.H; S10 §§5.5–5.8

T09 — Software verification and validation (2/2)

Applies: Apply the established documentation route. S1 §VI.H covers system protocols/results at Basic level and additionally unit/integration protocols/reports at Enhanced level; use targeted detail for the review question.

□ T09.07 Follow failed and blocked tests to disposition

Where: Failure log; deviation register; anomaly list · **Search:** failed, blocked, deviation, anomaly ID

Why: Aggregate pass statements can conceal unresolved findings.

Details to check: Failed criterion; actual observation; cause/impact; affected version; linked anomaly; disposition; release implications.

□ T09.08 Check that claimed fixes have relevant retests

Where: Retest logs; fix records; test summary · **Search:** fixed, rerun, retest, resolved

Why: A fix claim does not supply verification of the fix.

Details to check: Original failure; corrective change; fixed build; retest method/result; affected coverage; remaining failures or accepted limitations.

□ T09.09 Check the regression-selection rationale

Where: Change-impact assessment; regression plan/report · **Search:** selected tests, regression, impact, excluded

Why: Repeating some tests may leave changed behavior uncovered.

Details to check: Changed functions/interfaces/dependencies; affected risks/requirements; tests selected and omitted; rationale; executed outcomes.

□ T09.10 Check whether the release conclusion matches the evidence

Where: V&V conclusion; release review; unresolved-item register · **Search:** ready for release, conditional, deferred, limitation

Why: A favorable conclusion may depend on unfinished work.

Details to check: Supported version/scope; unresolved failures/anomalies; exclusions; residual conditions; referenced evidence actually available; future commitments.

Related checks: T03, T04, T05, T10, T11, T23 | Source pointers: S1 §VI.H; S10 §§5.5–5.8

T10 — Version history and changes

Applies: Apply when earlier versions, prior authorizations or changed configurations support the proposed release.

□ T10.01 Find the version sequence and associated evidence

Where: Version history; release notes; test/study reports · **Search:** version history, clinical version, last tested

Why: Identifies which version each result supports.

Details to check: Version/build sequence; dates; material changes; test, bench and clinical evidence associated with each baseline.

□ T10.02 Identify differences from the last tested version

Where: Change log; configuration comparison; release manifest · **Search:** change ID, from version, to version

Why: A release label can conceal untested modifications.

Details to check: Exact changed items/functions; dependencies and settings; previous/new versions; implemented versus planned changes.

□ T10.03 Check safety, security and performance impact

Where: Change-impact analysis; revised risk file · **Search:** no impact, affected control, new risk

Why: A no-impact assertion needs a supporting scope and rationale.

Details to check: Affected requirements/interfaces/controls; new or changed risks; downstream effects; basis for no-impact conclusions.

□ T10.04 Find evidence supporting reuse of earlier results

Where: Bridging rationale; equivalence analysis; regression report · **Search:** equivalent, bridge, unchanged, retest

Why: Earlier results need a defensible link to current behavior.

Details to check: Differences evaluated; unchanged evidence scope; technical rationale; repeated activities; actual results; residual limitations.

□ T10.05 Check previously authorized and study versions separately

Where: Prior-submission references; study configuration; current labeling · **Search:** authorized version, study build, marketed

Why: Authorization and performance evidence may concern different baselines.

Details to check: Prior decision scope; study/software configuration; candidate release; changed intended functions; bridge to each relied-on evidence source.

Related checks: T03, T04, T09, T16, T23 | Source pointers: S1 §VI.I; S10 §§6.2–6.3, 7.4, 8.2

T11 — Unresolved software anomalies

Applies: Apply to unresolved anomalies relevant to the proposed release and its evidence snapshot.

□ T11.01 Identify the unresolved-anomaly population

Where: Anomaly list; defect-register export; release review • **Search:** unresolved, open, query date, severity count

Why: A few mentions do not establish the complete supplied list.

Details to check: Software/build; list date; query filters; included statuses/severities; reported counts; received entries; scope exclusions.

□ T11.02 Check the description and affected scope of each material anomaly

Where: Individual defect records; reproduction notes • **Search:** anomaly ID, observed behavior, affected version

Why: A short label may hide the relevant failure mechanism.

Details to check: Unique ID; expected/actual behavior; trigger; discovery context; affected versions/configurations; investigation/root cause where available.

□ T11.03 Check safety and effectiveness impact

Where: Anomaly assessment; hazard analysis; release rationale • **Search:** impact, severity, hazard, unacceptable

Why: Low defect priority does not by itself establish low device risk.

Details to check: Severity taxonomy; patient/user consequences; linked risks; affected function; evidence and assumptions supporting the assessment.

□ T11.04 Check security implications separately

Where: Security anomaly assessment; vulnerability register • **Search:** security impact, weakness, vulnerability

Why: A functional defect can also expose an attack path.

Details to check: Security review criterion; relevant weakness; exploitability/impact rationale; linked vulnerability/control; unresolved consequences; see T18.

□ T11.05 Check disposition, workaround and communication

Where: Release review; anomaly disposition; IFU/advisory • **Search:** accepted, fixed, deferred, workaround, disclosed

Why: Different disposition labels have different release implications.

Details to check: Current status; effective version/date; justification; planned correction; practical workaround; user communication; remaining risk.

□ T11.06 Reconcile anomalies with test failures and release statements

Where: V&V failures; anomaly list; version history • **Search:** deferred test, failure ID, closed, release

Why: The same issue may have inconsistent states across documents.

Details to check: Matching IDs; compatible snapshots/configurations; failed/deferred tests; fixes/retests; related entries; explanation of apparent omissions or status conflicts.

Related checks: T04, T09, T10, T18, T29 | Source pointers: S1 §VI.J; S10 §§5.8.2–5.8.3, 9

T12 — Third-party / SOUP / off-the-shelf software

Applies: Apply to externally developed software relied upon by the device, using the relevant OTS/SOUP evidence route.

□ T12.01 Identify each relevant external component and role

Where: OTS/SOUP assessment; component register; SBOM • **Search:** supplier, component, version, patch, OTS, SOUP

Why: Suitability evidence must concern the installed component.

Details to check: Name; supplier; exact version/patch; function; device/software item; risk-control role; dependencies; classification used by the sponsor.

□ T12.02 Check suitability and known limitations

Where: Component assessment; supplier specifications • **Search:** suitability, limitation, restriction, intended use

Why: General supplier capability may not support device-specific use.

Details to check: Required behavior; operating limits; design limitations; known restrictions; intended device use; rationale and supporting evidence.

□ T12.03 Check required environment and permitted configurations

Where: Installation guide; compatibility matrix; configuration controls • **Search:** supported OS, permitted version, user configurable

Why: User changes can invalidate assessed component combinations.

Details to check: Hardware/OS requirements; compatible dependencies; permitted versions; configurable options; installation safeguards; prevention or handling of unsupported combinations.

□ T12.04 Find the version-specific known-anomaly review

Where: Supplier anomaly lists; issue tracker review; OTS assessment • **Search:** known bug, anomaly source, review date

Why: A general component review can miss version-relevant problems.

Details to check: Sources and query/review dates; component version; relevant anomalies; device impact; linked hazards/controls; disposition.

□ T12.05 Check device-level verification of relied-on behavior

Where: Integration tests; component tests; supplier reports • **Search:** integration, verified, supplier test, limitation

Why: Supplier testing may cover a different use or environment.

Details to check: Required behavior and risk controls; actual device configuration; test methods/results; limits of supplier evidence; remaining integration gaps.

□ T12.06 Check support and replacement arrangements

Where: Supplier lifecycle notice; maintenance plan; replacement assessment • **Search:** end of support, patch, replacement, continuity

Why: Device support can outlast component support.

Details to check: Component support date; device support horizon; monitoring/update responsibility; replacement or other mitigation; verification of changes; communicated constraints.

Related checks: T03, T04, T09, T16, T17 | Source pointers: S2 §III; S10 §§5.3.3–5.3.4, 7.1.3

T13 — Cybersecurity risk management process and report

Applies: Apply to the security-risk process for the device and the connected system within its defined scope.

□ T13.01 Find the security-risk plan and report scope

Where: Cybersecurity plan/report; system description • **Search:** scope, covered system, version, approval

Why: A conclusion may exclude a relevant system element.

Details to check: Document revisions; approval; device/system configuration; included assets/services; exclusions; lifecycle phases; relationship to the proposed release.

□ T13.02 Check the security-risk method and decision criteria

Where: Risk methodology; scoring procedure; assessment rubric • **Search:** acceptability, method, likelihood, residual

Why: Scores and conclusions depend on their underlying method.

Details to check: Evaluation method; criteria; assumptions; treatment of exploitability; risk acceptance roles; distinction between attack likelihood and patient-harm probability.

□ T13.03 Check ownership and supplier responsibilities

Where: Responsibility matrix; supplier agreement; security plan • **Search:** responsible, qualification, supplier, agreement

Why: Allocating a task does not demonstrate it was completed.

Details to check: Responsible roles and expertise; manufacturer/supplier boundaries; deliverables; escalation; evidence of completed responsibilities where claimed.

□ T13.04 Find the interface with safety risk management

Where: Security-to-safety mapping; safety risk file • **Search:** harm, safety transfer, patient impact

Why: Technical security consequences can become device hazards.

Details to check: Security event and affected function; hazardous situation/harm; linked safety-risk ID; transferred assumptions; corresponding controls and decisions.

□ T13.05 Check the basis of the overall security conclusion

Where: Security-risk report; open-findings register; release review • **Search:** residual risk, acceptable, unresolved, excluded

Why: An overall statement can obscure accepted or unassessed risks.

Details to check: Supporting assessments/tests; open findings; exclusions; external controls; accepted conditions; relevant release; support/end-of-service assumptions.

□ T13.06 Check software support categories when IEC 81001-5-1 is claimed

Where: Component classification; supplier responsibilities; risk-transfer records • **Search:** required, supported, maintained, transitional

Why: Category and scope determine the stated maintenance responsibilities.

Details to check: Component category; mapping of alternative terminology; manufacturer/operator duties; category changes and transferred risks; distinction from transitional status of the whole product.

Related checks: T04, T14, T15, T23, T25 | Source pointers: S3 §V.A; S13 §§4.1–4.3, 7; S17

T14 — Threat modeling

Applies: Apply to the actual device system, including relevant external services and lifecycle states.

□ T14.01 Identify the modeled system and method

Where: Threat-model report; architecture diagrams • **Search:** threat model, method, configuration, revision

Why: A generic threat list may not represent this device.

Details to check: Model date/revision; method and rationale; device/build scope; assets and security properties; included components and external systems.

□ T14.02 Check entry points and trust boundaries

Where: Data-flow diagrams; interface inventory; threat model • **Search:** trust boundary, entry point, interface, data flow

Why: Unmodeled connections can leave attack paths unexplored.

Details to check: Interfaces/endpoints; protocols; data direction; privilege boundaries; external dependencies; correspondence with deployed architecture.

□ T14.03 Follow threats through attack paths

Where: Threat tables; attack trees; misuse scenarios • **Search:** threat ID, attack vector, target, precondition

Why: Threat names alone do not explain how harm could arise.

Details to check: Threat ID; attacker capability/access; prerequisites; attack sequence; target asset; resulting impact; linked risk.

□ T14.04 Check environmental assumptions and exclusions

Where: Threat-model assumptions; deployment guide; labeling • **Search:** trusted network, assumption, excluded, transfer

Why: A control argument may rely on conditions outside the device.

Details to check: Each material assumption; justification or enforcement; external responsible actor; communication; consequences if violated; excluded-system rationale.

□ T14.05 Check coverage beyond normal operation

Where: Lifecycle/state diagrams; update and service threat models • **Search:** startup, update, recovery, service, decommission

Why: Threat coverage can change across lifecycle states.

Details to check: Relevant states; dormant/service interfaces; changed privileges; update/recovery paths; state-specific threats; scope of any exclusions.

□ T14.06 Follow threats to controls, tests or explicit disposition

Where: Threat-control matrix; risk assessment; security test plan • **Search:** mitigated by, control ID, test ID, unresolved

Why: A modeled threat can remain without supporting mitigation evidence.

Details to check: Threat/control/requirement/test links; implemented version; executed results; residual threats; rationale for accepted or unmitigated paths.

Related checks: T06, T15, T19, T21, T22, T23 | Source pointers: S3 §V.A.1 and Appendix 2; S13 §§5.3, 7.2

T15 — Cybersecurity risk assessment

Applies: Apply to individual security risks and the residual conditions relied on for the release.

□ T15.01 Identify the risk and affected device scope

Where: Security risk register; vulnerability assessment • **Search:** risk ID, threat, weakness, affected component

Why: A vulnerability score may concern a different configuration.

Details to check: Risk/vulnerability ID; linked threat/weakness; component and version; exposed function; affected release population; device/system consequences.

□ T15.02 Check the scoring basis and assumptions

Where: Risk worksheet; CVSS/rubric documentation • **Search:** vector, version, pre-mitigation, threshold

Why: Different rubrics or assumptions yield incomparable scores.

Details to check: Scoring system/version; vector or factors; device-specific rubric; criteria; exploit prerequisites; initial and residual assumptions.

□ T15.03 Check what the mitigation actually changes

Where: Risk-control rationale; architecture; mitigation assessment • **Search:** compensating control, post-mitigation, attack path

Why: A lower score should have an explained mechanism.

Details to check: Control and affected attack path; conditions for risk reduction; implementation scope; remaining exposure; changed score factors.

□ T15.04 Find effectiveness evidence for the mitigation

Where: Security tests; configuration verification; retest report • **Search:** effectiveness, retest, verified, bypass

Why: A revised score is not proof of an effective control.

Details to check: Control-specific test; relevant build/conditions; actual result; attempts to bypass or defeat the control where relevant; unresolved limitations.

□ T15.05 Check residual and newly introduced risks

Where: Residual-risk review; updated threat model • **Search:** residual, new risk, accepted, release condition

Why: Mitigation can leave or introduce material concerns.

Details to check: Remaining risk; applicable criterion; acceptance rationale; new risks; effective version/date; linked evidence and unresolved findings.

□ T15.06 Check transferred risk and external controls

Where: Risk-transfer agreement; IFU; service/support plan • **Search:** user responsibility, external control, end of service

Why: Risk remains dependent on another actor's actions.

Details to check: Recipient; required action/capability; conditions; communicated instructions; feasibility/evidence; duration; residual responsibility at support end.

Related checks: T04, T13, T17, T21, T23, T24 | Source pointers: S3 §V.A.2; S13 §7

T16 — Software bill of materials

Applies: Apply to the software/build and system-element scope represented by the SBOM.

□ T16.01 Identify the SBOM artifact and build scope

Where: SBOM file; explanatory report; build manifest · **Search:** SBOM, SPDX, CycloneDX, build, generation

Why: Generation date alone does not establish release identity.

Details to check: Format/specification version; author; generation tool/date; described product/build; connection to the proposed configuration.

□ T16.02 Check component identity fields

Where: SBOM entries; component register · **Search:** supplier, name, version, identifier, hash

Why: Ambiguous names prevent reliable component matching.

Details to check: Component name and supplier; exact version/patch; identifiers/hashes when used; relationship to the installed package; unresolved identity gaps.

□ T16.03 Check dependency and system coverage

Where: SBOM dependency graph; architecture; manifests · **Search:** dependency, transitive, excluded, cloud

Why: Top-level inventory may omit relevant nested or external software.

Details to check: Direct/transitive dependencies; relationships; device/cloud/intermediary scope; commercial/open-source/off-the-shelf components; omissions and rationale.

□ T16.04 Check support and lifecycle information

Where: SBOM support fields; supplier notices; support annex · **Search:** supported, end of support, end of life

Why: Inventory alone does not establish maintainability.

Details to check: Support status; end dates; affected component versions; source/date for lifecycle information; replacement or maintenance arrangements.

□ T16.05 Match SBOM entries to vulnerability review

Where: Component-to-vulnerability mapping; scan scope · **Search:** CVE, component match, query date, unmapped

Why: A scan can miss components absent from its matching scope.

Details to check: Same component/version identity; databases and query dates; matching method; unassessed or unmatched components; known-vulnerability disposition; see T17.

Related checks: T03, T12, T17, T19, T24 | Source pointers: S3 §V.A.4

T17 — Vulnerabilities and component support

Applies: Apply to known-vulnerability and component-support assessments for the relevant release population.

□ T17.01 Check the vulnerability-search scope and currency

Where: Scan report; database query record; component review • **Search:** database, query date, KEV, scan scope

Why: An empty search result is bounded by its date and coverage.

Details to check: Tools/databases and versions; query date; component population; matching rules; exploited-vulnerability checks; exclusions or inaccessible sources.

□ T17.02 Match each material finding to the installed component

Where: CVE/component mapping; SBOM; build configuration • **Search:** CVE, affected version, package, installed

Why: A named vulnerability may concern a different component variant.

Details to check: Vulnerability ID; component/supplier/version; affected range; installed configuration; mapping basis; unresolved identity ambiguity.

□ T17.03 Check device-specific exploitability and impact

Where: Vulnerability assessment; architecture; threat model • **Search:** not exploitable, disabled, prerequisite, impact

Why: Generic severity does not resolve device-specific exposure.

Details to check: Reachable interface; enabled feature; attacker access and prerequisites; affected function; safety/security consequences; evidence supporting dismissal.

□ T17.04 Check mitigations and residual disposition

Where: Mitigation record; risk assessment; retest • **Search:** fixed, compensating control, accepted, deferred

Why: Disposition depends on effective action and its conditions.

Details to check: Mitigation; implemented release; verification/retest; external reliance; residual risk; rationale for acceptance/deferral; planned correction.

□ T17.05 Identify affected releases and unsupported components

Where: Fielded-version inventory; supplier notices; support plan • **Search:** affected releases, unsupported, end of life

Why: A supported candidate may coexist with vulnerable fielded versions.

Details to check: Device/component versions; support end dates; deployed population; update/replacement coverage; unresolved gaps; user communication.

Related checks: T12, T15, T16, T23, T25 | Source pointers: S3 §§V.A.4, V.A.6; S13 §§6.2, 9

T18 — Security implications of unresolved anomalies

Applies: Apply to unresolved software anomalies that could have security consequences.

□ T18.01 Check that security review covers the relevant anomaly list

Where: Anomaly export; security assessment · **Search:** anomaly ID, reviewed list, snapshot

Why: A security conclusion may omit later or differently filtered defects.

Details to check: List/version/date; inclusion criteria; reviewed IDs; unreviewed items; reconciliation with T11 and the release snapshot.

□ T18.02 Find the security impact of each material anomaly

Where: Security anomaly analysis; threat/risk records · **Search:** security impact, denial of service, weakness, CWE

Why: A tolerable functional defect can enable deliberate misuse.

Details to check: Trigger; attacker influence; affected security property; weakness classification where used; device/system consequences; linked threats/risks.

□ T18.03 Check the rationale for vulnerability classification

Where: Anomaly-to-vulnerability decision; assessment criteria · **Search:** vulnerability, no security impact, not exploitable

Why: A classification label needs a device-specific basis.

Details to check: Decision criteria; rationale; configuration assumptions; supporting evidence; relationship to known vulnerabilities and attack paths.

□ T18.04 Follow security-relevant anomalies to controls and verification

Where: Vulnerability register; change record; security retest · **Search:** mitigation, retest, residual, accepted

Why: Safety disposition alone may leave security consequences unresolved.

Details to check: Shared anomaly identity; security control/fix; effective release; verification; residual consequences; acceptance/deferral rationale and communication.

Related checks: T11, T15, T17, T23 | Source pointers: S3 §V.A.5; S13 §§5.8.5, 9

T19 — Cybersecurity traceability

Applies: Apply to source-supported links within the relevant security evidence snapshot.

□ T19.01 Identify the traceability export and namespaces

Where: Trace matrix; requirements-tool export • **Search:** export date, baseline, ID, revision

Why: Identical-looking IDs can refer to different objects or versions.

Details to check: Export date; system/build; object types; ID namespaces; document revisions; included and excluded populations.

□ T19.02 Follow threats and risks to requirements and controls

Where: Threat-risk-control matrix; security requirements • **Search:** mitigated by, requirement ID, control ID

Why: A risk can lack an implemented treatment despite a populated table.

Details to check: Actual source-authored links; threat/risk identity; control/requirement text; accepted exceptions; missing or contradictory targets.

□ T19.03 Follow controls to architecture and affected components

Where: Architecture mapping; component/SBOM links • **Search:** architecture element, component ID, implements

Why: A generic control may not protect the relevant asset or interface.

Details to check: Implementing element; protected asset/interface; component/version; configuration; matching names/IDs across views.

□ T19.04 Follow controls to executed tests

Where: Trace matrix; test definitions; run results • **Search:** test case, run ID, verified, result

Why: A link to a case is not evidence of execution.

Details to check: Control/requirement ID; test ID; execution record; tested build; actual outcome; unexecuted or failed tests.

□ T19.05 Check reverse coverage and dangling links

Where: Reverse trace export; linked source records • **Search:** untraced, orphan, missing target, bidirectional

Why: Selected forward links cannot establish complete coverage.

Details to check: Scope actually checked; controls/tests without a supported source; unresolved targets; references whose contents are unavailable; version mismatches.

Related checks: T05, T14, T15, T16, T21, T23 | Source pointers: S3 §§V.A–V.C and Appendix 2; S13 §§5.2–5.7

T20 — Cybersecurity measures and metrics

Applies: Apply where security-maintenance metrics are supplied or relevant to stated commitments; distinguish new-product data limits.

□ T20.01 Find each metric's definition and population

Where: Metrics report; dashboard specification • **Search:** definition, denominator, reporting period

Why: A percentage is uninterpretable without its population.

Details to check: Numerator/denominator; device/version population; reporting period; units; inclusions/exclusions; source of the figures.

□ T20.02 Check the identification-to-patch interval

Where: Vulnerability/patch register; metrics report • **Search:** identification, discovery, confirmation, patch release

Why: Different starting events produce different timing claims.

Details to check: Start/end event definitions; dates; unit; affected vulnerabilities; calculation method; exceptions; distinction from deployment time.

□ T20.03 Check patch-to-deployment and field coverage

Where: Deployment report; fielded-version inventory • **Search:** deployed, installed, fielded, percentage patched

Why: Releasing a patch does not show that devices received it.

Details to check: Patch-release versus installation dates; eligible/updated population; affected versions; deployment evidence; offline or excluded devices.

□ T20.04 Check zero, unavailable and nonapplicable results

Where: Metric notes; supporting register; plan • **Search:** zero, no data, unavailable, not applicable

Why: These states imply different evidence and limitations.

Details to check: Explicit value and reason; actual data availability; product/lifecycle context; calculation scope; planned measurement where relevant.

□ T20.05 Compare metrics with management commitments

Where: Security-management plan; update policy; metrics • **Search:** target, routine, exceptional, service level

Why: Apparent performance can depend on incompatible clocks or scope.

Details to check: Same trigger, interval and population; stated target; reported result; missed targets; explanations; version-specific risk context.

Related checks: T17, T23, T25 | Source pointers: S3 §V.A.6

T21 — Security controls and requirements (1/2)

Applies: Select controls relevant to the device assets, interfaces, threats and deployment conditions; do not assume every mechanism applies.

☐ T21.01 Check authentication of users, devices and services

Where: Authentication design; interface specification; security tests • **Search:** authentication, identity, credential, default password

Why: Each trusted entity needs a supported identity mechanism.

Details to check: Entity/interface covered; mechanism; initial credentials; credential changes/revocation; default/hardcoded credentials; failed-authentication behavior; verification.

☐ T21.02 Check authorization and privilege boundaries

Where: Role/permission matrix; access-control design; tests • **Search:** role, authorization, least privilege, service access

Why: An authenticated entity may still have excessive access.

Details to check: Roles; permitted actions/resources; privileged/service functions; enforcement points; privilege changes; denied-action tests; exceptional access.

☐ T21.03 Check cryptographic protection of relevant flows and assets

Where: Crypto specification; security architecture; test report • **Search:** algorithm, protocol, mode, key length

Why: An algorithm name alone leaves protection scope unclear.

Details to check: Protected data/flow; algorithm/protocol/version; mode/settings; key strength; endpoints; negotiated or fallback behavior; implementation/test evidence.

☐ T21.04 Check key and certificate management

Where: Key-management design; provisioning/update procedures • **Search:** key generation, storage, certificate, revocation

Why: Strong algorithms depend on protected, managed secrets.

Details to check: Key creation/distribution; storage and access; device uniqueness/shared-key scope; replacement/revocation; certificate validation; responsibility and verification.

☐ T21.05 Check integrity of code and data

Where: Boot/update design; data-integrity requirements; tests • **Search:** signature, integrity, authenticity, tamper

Why: Corrupted or substituted content can affect device behavior.

Details to check: Protected code/data; origin and integrity checks; verification stage; rejection behavior; relevant version metadata; tamper/failure test results.

☐ T21.06 Check confidentiality of sensitive data

Where: Data-flow inventory; storage/transport design • **Search:** confidential, at rest, in transit, privacy

Why: Protection can stop at an intermediate system or storage location.

Details to check: Sensitive assets; storage/transmission paths; authorized access; protection endpoints; intermediaries; retention/deletion where relevant; verification scope.

Related checks: T05, T14, T15, T19, T22, T23 | Source pointers: S3 §V.B.1 and Appendix 1; S13 §§5.2–5.7

T21 — Security controls and requirements (2/2)

Applies: Select controls relevant to the device assets, interfaces, threats and deployment conditions; do not assume every mechanism applies.

□ T21.07 Check security event detection and logs

Where: Logging specification; administrator guide; tests · **Search:** event, audit log, timestamp, retention

Why: Users may rely on logs that omit decisive events.

Details to check: Events captured; timestamp/time source; fields/location; access and tamper protection; retention; alerting/review actions; behavior if logging fails.

□ T21.08 Check safe behavior during security failures

Where: Resilience design; denial-of-service tests; hazard analysis · **Search:** safe state, degraded, availability, resource exhaustion

Why: Security controls can impair essential device functions.

Details to check: Relevant failure/attack conditions; maintained functions; degraded/safe state; resource limits; recovery criteria; safety links; actual tests.

□ T21.09 Check backup and recovery

Where: Backup/restore procedure; recovery tests · **Search:** backup, restore, recovery, configuration

Why: Recovery must restore a usable and controlled state.

Details to check: Data/configuration covered; protection; access; restore prerequisites; restored version/state; failed-restore handling; demonstrated recovery.

□ T21.10 Check update authenticity and interruption handling

Where: Update design; rollback/recovery tests · **Search:** signed update, rollback, interruption, partial install

Why: An update may leave untrusted or inconsistent software.

Details to check: Update source and authentication; compatibility/version checks; interrupted-install state; rollback policy; recovery; signing-key protection; actual failure-path tests.

Related checks: T05, T14, T15, T19, T22, T23 | Source pointers: S3 §V.B.1 and Appendix 1; S13 §§5.2–5.7

T22 — Security architecture views

Applies: Check architecture views appropriate to the device system and relevant security use cases.

□ T22.01 Find the global system view

Where: Security architecture; deployment diagrams • **Search:** global view, cloud, intermediary, network

Why: Shows the complete operating system and its trust boundaries.

Details to check: Device and external elements; owners; trust boundaries; assets; dependencies; diagram revision and configuration; consistency with T06.

□ T22.02 Check potential effects across multiple devices or patients

Where: Multi-patient view; fleet/cloud architecture; threat model • **Search:** multi-patient, shared service, fleet, propagation

Why: A shared dependency can expand the impact of compromise.

Details to check: Shared assets/credentials/services; paths between devices; scale of effect; isolation and containment controls; rationale if this view is inapplicable.

□ T22.03 Follow update and patch delivery end to end

Where: Update architecture; distribution/service diagrams • **Search:** update server, distribution, signing, patch path

Why: Protection may fail outside the device's local updater.

Details to check: Source through intermediaries to device; ownership/trust; payload and metadata; authentication; delivery states; verification and recovery points.

□ T22.04 Check security use cases across operating states

Where: Security use-case views; state diagrams • **Search:** programming, alarming, standby, service, recovery

Why: Clinical and operational states can expose different paths.

Details to check: Relevant state and clinical function; end-to-end communication; enabled/dormant interfaces; privileges; controls; safety consequences; exclusions.

□ T22.05 Check the detail on each relevant communication path

Where: Interface/port tables; diagram annotations • **Search:** endpoint, port, protocol, session, default

Why: A line between boxes omits the facts needed to assess protection.

Details to check: Endpoints; direction; payload; protocol/version/port; authentication; cryptography; session handling; defaults; linked control/test/component IDs.

Related checks: T06, T14, T19, T21, T23, T24 | Source pointers: S3 §V.B.2 and Appendix 2

T23 — Cybersecurity testing (1/2)

Applies: Select test activities against the applicable security requirements, threat model and device boundaries.

☐ T23.01 Identify test activities, objectives and boundaries

Where: Security test plan; report scope; architecture • **Search:** scope, objective, exclusion, test type

Why: Different methods address different security questions.

Details to check: Requirements, threats and interfaces covered; excluded systems; rationale; test activity type; connection to the threat model.

☐ T23.02 Check the tested configuration and tool settings

Where: Report header; environment section; tool output • **Search:** build, tool version, configuration, database date

Why: Results depend on the release, environment and test settings.

Details to check: Software/build; components; enabled interfaces; environment; tool versions/settings; vulnerability-source dates; test dates; release bridge.

☐ T23.03 Check security-requirement and threat-mitigation tests

Where: Requirements tests; threat-driven tests; control matrix • **Search:** requirement ID, mitigation, boundary, effectiveness

Why: Implementing a control does not demonstrate resistance to the modeled threat.

Details to check: Requirement/control/threat IDs; method; conditions; boundary assumptions; criterion; actual result; failed or excluded cases.

☐ T23.04 Check vulnerability-testing methods and coverage

Where: Scan/analysis reports; robustness/fuzz tests • **Search:** malformed input, fuzz, attack surface, static, dynamic

Why: A single scan does not cover all relevant weaknesses.

Details to check: Methods and rationale; attack-surface coverage; scanning, static/dynamic and binary composition analysis; malformed-input/fuzz testing; vulnerability chaining where relevant; target interfaces/components; tool scope; findings and exclusions.

☐ T23.05 Check penetration-test expertise, independence and effort

Where: Original penetration-test report; tester qualifications • **Search:** penetration, independence, expertise, duration

Why: The strength of an exercise depends on who tested what and how.

Details to check: Tester relationship to developers; qualifications; duration/effort; access/knowledge assumptions; methods; tested scope; original report availability.

☐ T23.06 Find the actual findings and observations

Where: Original test reports; findings register; evidence annexes • **Search:** finding ID, observation, severity, evidence

Why: A favorable executive statement can omit actionable findings.

Details to check: Finding IDs; evidence; severity method; affected asset/build; reproduction conditions; reported counts; linked risk assessment.

Related checks: T03, T14, T15, T17, T19, T21, T25 | Source pointers: S3 §V.C; S13 §§5.7–5.8

T23 — Cybersecurity testing (2/2)

Applies: Select test activities against the applicable security requirements, threat model and device boundaries.

☐ **T23.07 Check fixes against relevant retest results**

Where: Remediation register; retest report; fixed-build record · **Search:** corrected, fixed build, retested, remaining

Why: A closed finding may have no demonstrated correction.

Details to check: Finding-to-fix link; changed build; retest scope/method; actual result; regression effects; findings not retested.

☐ **T23.08 Check accepted, deferred and excluded findings**

Where: Residual-risk review; test conclusion; release plan · **Search:** accepted, deferred, excluded, post-release

Why: Remaining findings can qualify the release conclusion.

Details to check: Each material disposition; risk rationale; compensating controls; affected release; user actions; future commitments; timing and verification plan.

Related checks: T03, T14, T15, T17, T19, T21, T25 | Source pointers: S3 §V.C; S13 §§5.7–5.8

T24 — Cybersecurity labeling

Applies: Apply to security-related instructions and disclosures needed for the intended users, administrators and service personnel.

□ T24.01 Check deployment prerequisites and recommended controls

Where: IFU security section; administrator guide; supplied disclosure questionnaire · **Search:** secure network, firewall, endpoint, ports

Why: Risk arguments may depend on the user's environment.

Details to check: Required/recommended infrastructure; network diagram; ports/protocols/direction; approved endpoints; controls users must configure; responsibility boundaries.

□ T24.02 Check version identification and SBOM access

Where: IFU; version screen; SBOM delivery instructions · **Search:** version, build, SBOM, portal

Why: Users need information for their actual device configuration.

Details to check: How to identify software; matching SBOM version; delivery/access method; intended recipient; actual artifact versus a portal reference.

□ T24.03 Check incident recognition and response instructions

Where: Security manual; incident-response section · **Search:** security event, log, incident, contact

Why: Detection features are useful only if users can act on them.

Details to check: Event/alert meaning; log location/format; user actions; escalation/contact; evidence preservation; safe operation during an incident.

□ T24.04 Check secure setup, backup and recovery instructions

Where: Installation guide; backup/restore procedure · **Search:** secure default, backup, restore, update

Why: Reliance on administrative action needs usable instructions.

Details to check: Initial configuration; account/access setup; backup protection; recovery steps; update process; failure handling; role-specific responsibilities.

□ T24.05 Check support dates and component limitations

Where: IFU support section; lifecycle notices; supplier annex · **Search:** support end, component support, end of service

Why: Device and component support promises may differ.

Details to check: Device/version horizon; relevant component end dates; replacement/update coverage; support-end actions; consistency with SBOM and management plan.

□ T24.06 Check risk transfers, unsafe changes and decommissioning

Where: Warnings; security limitations; sanitization procedure · **Search:** risk transfer, configuration change, sanitize, retire

Why: General warnings may omit actions needed for the risk argument.

Details to check: Specific user controls; changes increasing risk; residual risks; service/retirement steps; data/credential sanitization; feasibility and human-factors links.

Related checks: T15, T16, T21, T22, T25, T27, T29 | Source pointers: S3 §VI.A and Appendix 1; S13 §5.8.2 and Annex E

T25 — Postmarket cybersecurity management

Applies: Apply to the security-management commitments for supported and fielded device versions.

☐ T25.01 Find ownership and vulnerability monitoring arrangements

Where: Postmarket security plan; PSIRT procedure • **Search:** PSIRT, monitoring, source, frequency

Why: Detection depends on assigned and sustained activity.

Details to check: Responsible roles; monitoring sources/frequency; component/SBOM linkage; affected-product identification; escalation and resourcing.

☐ T25.02 Check vulnerability disclosure and intake

Where: Disclosure policy; intake procedure; public contact • **Search:** coordinated disclosure, intake, advisory

Why: External findings need a defined path into action.

Details to check: Reporting channels; triage; communication responsibilities; coordination; advisory contents; tracking through disposition.

☐ T25.03 Check routine and exceptional patch commitments

Where: Patch policy; management plan • **Search:** routine, out-of-cycle, trigger, timeframe

Why: Different risk situations can require different update paths.

Details to check: Trigger definitions; timelines and units; prioritization; verification; approval; exceptional handling; distinction between planned targets and completed action.

☐ T25.04 Check deployment to fielded versions

Where: Update architecture; field-service plan; version inventory • **Search:** deployment, fielded, offline, supported

Why: A verified patch may not reach affected devices.

Details to check: Supported release population; delivery mechanism; installation responsibility; compatibility; deployment confirmation; offline/failed updates; communications.

☐ T25.05 Check support and end-of-service transitions

Where: Support policy; end-of-service plan; IFU • **Search:** supported, maintained, end of service, transfer

Why: Risk can remain after routine maintenance ends.

Details to check: Product/component horizons; notification; affected fielded versions; replacement/retirement options; residual risks; transferred responsibilities.

☐ T25.06 Compare commitments with available execution evidence

Where: Metrics; incident/change records; prior updates • **Search:** completed, deployed, target, overdue

Why: A plan and a completed activity support different claims.

Details to check: Relevant event/version; stated target; actual action/date; exceptions; unresolved commitments; standards/process claims and their stated scope.

Related checks: T17, T20, T23, T24, T26 | Source pointers: S3 §VI.B; S13 §§4.1.7–4.1.8, 6, 9; S16 §§V–VI

T26 — Cyber-device applicability statements

Applies: Check factual characteristics and the applicability rationale against the relevant legal criteria; an unchecked box is not a legal determination.

☐ T26.01 Find the stated cyber-device applicability decision

Where: Applicability statement; submission cover; legal-basis section • **Search:** cyber device, 524B, applicability

Why: Identifies the decision and evidence route being claimed.

Details to check: Applicable submission/device scope; stated conclusion; cited legal basis; date; underlying rationale; unresolved qualifications.

☐ T26.02 Check the software characteristic relied upon

Where: Device description; software inventory; architecture • **Search:** includes software, validated, installed, authorized

Why: The factual software scope must match the classification rationale.

Details to check: Included software; installation/validation/authorization relationship; device functionality; relevant components and boundaries.

☐ T26.03 Check internet-connectivity capability

Where: Interface specification; network/service architecture • **Search:** internet, indirect, optional, disabled, service port

Why: A marketing label may omit actual connectivity capability.

Details to check: Direct/indirect paths; connectors and counterpart systems; optional/service interfaces; enabled/disabled configurations; evidence supporting the claimed capability.

☐ T26.04 Check vulnerable technological characteristics

Where: Architecture; threat model; security assessment • **Search:** technological characteristic, vulnerability, susceptible

Why: Applicability needs the actual device characteristics.

Details to check: Relevant hardware/software features; exposure and vulnerability rationale; configuration assumptions; consistency with connectivity and software descriptions.

☐ T26.05 Locate the evidence relied on under the applicable route

Where: Applicability cross-references; management plan; SBOM • **Search:** monitoring plan, update process, SBOM

Why: A document reference alone supplies no examinable content.

Details to check: Actual monitoring/disclosure plan; update/security processes; SBOM; relevant device/version; specific applicability basis for each requested element.

Related checks: T02, T06, T16, T24, T25 | Source pointers: S3 §VII

T27 — Usability and human factors (1/2)

Applies: Use the justified human-factors submission route; examine validation details when applicable to the device, changes and use-related risks.

☐ T27.01 Find the user, environment and interface scope

Where: Use specification; device description; IFU • **Search:** user group, environment, UI, training

Why: Evidence must represent the people and conditions of use.

Details to check: Intended user groups; capabilities; environments; tasks; software/UI/labeling versions; training assumptions; known use problems.

☐ T27.02 Find the claimed submission category and rationale

Where: HF category assessment; change analysis • **Search:** category, rationale, new device, modified device

Why: The applicable evidence route depends on the device and changes.

Details to check: Claimed category; device/change scope; use-related risk reasoning; cited criteria; supporting analysis; unresolved applicability.

☐ T27.03 Identify critical tasks and their risk links

Where: Task analysis; use-related risk analysis • **Search:** critical task, use error, hazardous situation

Why: A study may omit the task that creates the risk.

Details to check: Task ID and sequence; foreseeable errors; resulting harm; critical-task rationale; linked controls; scenarios selected for evaluation.

☐ T27.04 Check formative findings and design responses

Where: Formative studies; design-change records • **Search:** formative, observed problem, design change

Why: Earlier use problems may remain in the final interface.

Details to check: Users/tasks studied; findings; selected design controls; changes made; verification/evaluation; issues carried into validation.

☐ T27.05 Check participant and scenario representativeness

Where: Validation protocol/report; recruitment criteria • **Search:** participant, group, scenario, sample rationale

Why: Completion counts are meaningful only for a relevant study.

Details to check: User groups and counts; selection rationale; task/scenario coverage; environments; actual versus planned conditions; exclusions and deviations.

☐ T27.06 Check training, assistance and final interface fidelity

Where: Study methods; moderator script; test configuration • **Search:** training, decay, prompting, assistance, final UI

Why: Assistance or unfamiliar configurations can distort observed performance.

Details to check: Training type/timing; user access to instructions; moderator intervention; UI/labeling/software versions; differences from marketed use.

Related checks: T02, T04, T09, T24, T29, T30 | Source pointers: S4 §§5–8; S5 §§IV–V; S12 §§5.1–5.9

T27 — Usability and human factors (2/2)

Applies: Use the justified human-factors submission route; examine validation details when applicable to the device, changes and use-related risks.

☐ T27.07 Find observed errors, close calls and difficulties

Where: Task-level results; observation logs; interviews • **Search:** use error, close call, difficulty, root cause

Why: Successful completion can conceal unsafe interaction.

Details to check: Task/user group; actual behavior; assistance; errors and near misses; causes; participant feedback; relationship to critical tasks.

☐ T27.08 Check residual use-risk reasoning and remaining changes

Where: HF conclusion; risk file; revised labeling/UI • **Search:** residual risk, unresolved, redesign, additional validation

Why: A favorable overall statement must address material findings.

Details to check: Finding-specific risk analysis; further design changes; justification for remaining risk; supporting evidence; need for further evaluation after changes.

Related checks: T02, T04, T09, T24, T29, T30 | Source pointers: S4 §§5–8; S5 §§IV–V; S12 §§5.1–5.9

T28 — Standards and conformity claims

Applies: Apply to each actual conformity, partial-use or equivalent-evidence claim; verify recognition for the relevant review date.

☐ T28.01 Identify the exact claim and product scope

Where: Declaration; standards list; submission statement • **Search:** declaration, conformity, general use, edition

Why: Mentioning a standard does not define the claimed evidence route.

Details to check: Exact standard/edition/amendment; full or selected-clause claim; device/software identity; scope/options/classes; declaration date and signatory if supplied.

☐ T28.02 Check recognition and transition details where relied upon

Where: FDA recognition entry; supplemental information • **Search:** recognition number, extent, transition, edition

Why: Recognition can differ by edition, clause and date.

Details to check: Dated official entry; recognized edition; extent/exclusions; supplemental information; transition conditions; match to the sponsor's actual claim.

☐ T28.03 Check exclusions, deviations and alternatives

Where: Clause matrix; declaration limitations; rationale • **Search:** excluded, deviation, not applicable, equivalent

Why: A broad claim can conceal important limitations.

Details to check: Specific clauses/options affected; reasons; device scope; alternative method/evidence; limits of the resulting conformity claim.

☐ T28.04 Locate the supporting evidence

Where: Referenced reports; test records; conformity attachments • **Search:** report number, evidence, completed, pending

Why: A declaration or citation does not supply all underlying facts.

Details to check: Report identity/revision; relevant contents; methods/results; completion versus future testing; access limits; relation to claimed clauses.

☐ T28.05 Check tested-product and edition consistency

Where: Original reports; declaration; release/change records • **Search:** tested version, edition, equivalent, bridge

Why: Evidence may concern a different product or standard edition.

Details to check: Tested device/software; report edition/options; marketed release; changes; equivalence/bridging rationale; unresolved mismatches.

☐ T28.06 Check laboratory claims where relied upon

Where: Laboratory report; accreditation certificate/scope • **Search:** accredited, scope, laboratory, method

Why: Accreditation claims are limited to a stated scope and time.

Details to check: Laboratory identity; relevant accreditation scope; validity at testing; method covered; report authenticity/provenance; limitations of the claim.

Related checks: T01, T03, T09, T23, T27 | Source pointers: S6 §§IV–VIII; S7

T29 — Software/device labeling

Applies: Apply to labeling supporting the reviewed software functions; use T24 for detailed cybersecurity instructions.

□ T29.01 Match labeling to the device and software release

Where: IFU cover/history; version screen; release manifest • **Search:** IFU revision, software version, About

Why: A current document revision can still describe an older release.

Details to check: Label/IFU revision; software identified; intended device/configuration; user-visible version; controlled draft/final status; discrepancies.

□ T29.02 Check operating instructions and limits

Where: IFU; technical description; training materials • **Search:** operating range, limitation, input, output

Why: User-facing claims should match requirements and validation.

Details to check: Intended users/use; input/output interpretation; operating limits; warnings; supported functions/configurations; consistency with tested conditions.

□ T29.03 Check permitted external components and configuration changes

Where: Installation guide; compatibility list; IFU • **Search:** permitted version, compatible, update, warning

Why: User-selected components can alter validated behavior.

Details to check: Allowed component/OS versions; installation or update restrictions; compatibility checks; user warnings; handling of unsupported combinations.

□ T29.04 Check anomaly and residual-risk disclosures

Where: IFU warnings; anomaly notice; risk-management report • **Search:** known limitation, anomaly, residual risk

Why: Accepted risk may depend on information reaching users.

Details to check: Relevant anomaly/risk; affected release; consequence; action/workaround; disclosure scope; consistency with accepted risk controls.

□ T29.05 Check instructions used as risk controls

Where: Risk-control matrix; IFU; human-factors evidence • **Search:** information for safety, warning, critical task

Why: Wording alone does not establish that the control works.

Details to check: Exact user action; intended user/environment; clarity and location; linked risk; effectiveness evidence; network-failure/security instructions where relevant.

Related checks: T02, T03, T04, T11, T24, T27 | Source pointers: S1 §§VI.B, VI.J; S4 §§5, 8; S11 §§7.3, 8

T30 — Machine-learning functions and controls (1/2)

Applies: Apply to machine-learning functions. PCCP checks apply when a plan is proposed or relied on; S9 is draft background, not a finalized requirement.

☐ T30.01 Identify the model and processing pipeline

Where: Model description; artifact manifest; software configuration • **Search:** model version, artifact hash, framework, locked

Why: Performance belongs to a particular model and pipeline.

Details to check: Model identity/version/hash where used; framework; preprocessing/postprocessing; decision settings; locked/adaptive behavior; proposed versus tested artifact.

☐ T30.02 Check data provenance and intended-population coverage

Where: Dataset description; provenance records; site tables • **Search:** dataset version, site, dates, population

Why: Validation may not represent intended use.

Details to check: Data sources; collection sites/time; selection criteria; relevant populations/conditions; dataset version; exclusions; relationship to intended use.

☐ T30.03 Check training, tuning and test independence

Where: Partition records; study design; data-flow description • **Search:** train, tune, test, overlap, external validation

Why: A split label does not rule out information leakage.

Details to check: Partition method; patient/site/time overlap; separation of development and testing; model-selection use; test reuse; external cohort identity where claimed.

☐ T30.04 Check reference labels and data handling

Where: Annotation protocol; reference-method description; data-quality report • **Search:** reference standard, ground truth, annotation, missing

Why: Performance depends on trustworthy targets and processing.

Details to check: Reference method; annotator/adjudication process where relevant; label quality; missing/invalid data; preprocessing; exclusions and their effect on evaluation.

☐ T30.05 Find performance targets and actual results

Where: Validation protocol; performance tables; statistical analysis • **Search:** metric, threshold, denominator, confidence interval

Why: An aggregate favorable metric can lack a justified target.

Details to check: Metric definition; acceptance criterion and rationale; tested cohort/model; sample counts; actual result; uncertainty; failed criteria; conditions.

☐ T30.06 Check subgroup, site and condition performance

Where: Stratified results; external-validation report • **Search:** subgroup, site, stratified, outlier

Why: Overall results can conceal important differences.

Details to check: Relevant subgroup/site definitions; denominators; estimates and uncertainty; exclusions; underrepresented groups; limitations on conclusions.

Related checks: T02, T03, T04, T09, T10, T27, T28 | Source pointers: S8 §§VI–VIII and Appendix A; S9 §§V–XII (draft background only)

T30 — Machine-learning functions and controls (2/2)

Applies: Apply to machine-learning functions. PCCP checks apply when a plan is proposed or relied on; S9 is draft background, not a finalized requirement.

□ T30.07 Check limitations and user interpretation

Where: Model labeling; transparency material; HF evidence · **Search:** limitation, uncertainty, explanation, human oversight

Why: Users need to understand the output's intended role and limits.

Details to check: Supported inputs/populations; known failure conditions; output interpretation; user actions; disclosures; consistency with observed performance.

□ T30.08 Check ML-specific safety and security controls

Where: ML risk assessment; robustness tests; monitoring plan · **Search:** data integrity, model integrity, drift, failure mode

Why: Data or model changes can alter device behavior.

Details to check: Relevant hazards/threats; model/data integrity controls; robustness evidence; monitoring triggers where relied upon; response actions; residual risks.

□ T30.09 Check proposed or authorized model changes

Where: PCCP; modification protocol; impact assessment · **Search:** PCCP, modification, retraining, acceptance, rollback

Why: A proposed plan does not establish authorization or completed validation.

Details to check: Plan status; specified modifications; data/retraining controls; evaluation criteria; update procedures; impact assessment; modification-to-test links; version control and user communication.

Related checks: T02, T03, T04, T09, T10, T27, T28 | Source pointers: S8 §§VI–VIII and Appendix A; S9 §§V–XII (draft background only)

Source pointers — 1

Use the sections cited with each topic. These are reading pointers for authored checks, not a clause-by-clause compliance checklist. Confirm the applicable edition, recognition and evidence route for the case.

S1 — Content of Premarket Submissions for Device Software Functions

June 2023; final guidance [Official source](#)

Local copy: `fda/Premarket-Software-Functions-Guidance.pdf`

S2 — Off-The-Shelf Software Use in Medical Devices

August 2023; final guidance [Official source](#)

Local copy: `fda/Off-The-Shelf-Guidance_0.pdf`

S3 — Cybersecurity in Medical Devices: Quality Management System Considerations and Content of Premarket Submissions

February 2026; final guidance [Official source](#)

Local copy: `fda/FDA-Cybersecurity-QMS-Premarket-2026.pdf`

S4 — Applying Human Factors and Usability Engineering to Medical Devices

August 2026; final guidance [Official source](#)

Local copy: `fda/FDA-Applying-Human-Factors-Usability-Engineering-2026.pdf`

S5 — Content of Human Factors Information in Medical Device Marketing Submissions

May 2026; final guidance [Official source](#)

Local copy: `fda/FDA-Content-Human-Factors-Information-Marketing-Submissions.pdf`

Local filenames identify copies in the sibling standards repository. Source publications remain there. Draft AI lifecycle guidance is background only. A process standard does not automatically require every internal record in a submission.

Source pointers — 2

Use the sections cited with each topic. These are reading pointers for authored checks, not a clause-by-clause compliance checklist. Confirm the applicable edition, recognition and evidence route for the case.

S6 — Appropriate Use of Voluntary Consensus Standards in Premarket Submissions for Medical Devices

September 2018; final guidance [Official source](#)

Local copy: fda/FDA-Appropriate-Use-Voluntary-Consensus-Standards-2018.pdf

S7 — FDA Recognized Consensus Standards Database

Live database; verify for the review date [Official source](#)

Local copy: fda/FDA-Recognized-Consensus-Standards-Database-page.html

S8 — Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions

August 2025; final guidance [Official source](#)

Local copy: fda/Guidance-Predetermined-Change-Control-AI.pdf

S9 — Artificial Intelligence-Enabled Device Software Functions: Lifecycle Management and Marketing Submission Recommendations

January 2025; DRAFT, not for implementation [Official source](#)

Local copy: fda/FDA-AI-Enabled-Device-Software-Functions-2025.pdf

S10 — IEC 62304 — Medical device software: Software life cycle processes

2006 + Amendment 1:2015; consolidated edition 1.1, final text

Local copy: iec62304{ed1.1}b.pdf

Local filenames identify copies in the sibling standards repository. Source publications remain there. Draft AI lifecycle guidance is background only. A process standard does not automatically require every internal record in a submission.

Source pointers — 3

Use the sections cited with each topic. These are reading pointers for authored checks, not a clause-by-clause compliance checklist. Confirm the applicable edition, recognition and evidence route for the case.

S11 — ISO 14971 — Medical devices: Application of risk management to medical devices

2019; third edition

Local copy: ISO+14971-2019.pdf

S12 — IEC 62366-1 — Application of usability engineering to medical devices

2015 + Amendment 1:2020; consolidated edition 1.1, final text

Local copy: iec62366-1{ed1.1}b.pdf

S13 — IEC 81001-5-1 — Security: Activities in the product life cycle

2021; edition 1.0; read with interpretation sheet S17

Local copy: IEC+81001-5-1-2021.pdf

S14 — Premarket Approval Application and Humanitarian Device Exemption Modular Review

January 2025; final guidance [Official source](#)

Local copy: fda/FDA-PMA-Modular-Review.pdf

S15 — Multiple Function Device Products: Policy and Considerations

July 2020; final guidance; local copy includes a QMSR notice [Official source](#)

Local copy: fda/FDA-Multiple-Function-Device-Products-2020.pdf

Local filenames identify copies in the sibling standards repository. Source publications remain there. Draft AI lifecycle guidance is background only. A process standard does not automatically require every internal record in a submission.

Source pointers — 4

Use the sections cited with each topic. These are reading pointers for authored checks, not a clause-by-clause compliance checklist. Confirm the applicable edition, recognition and evidence route for the case.

S16 — Postmarket Management of Cybersecurity in Medical Devices

December 2016; final guidance; local copy includes a QMSR notice [Official source](#)

Local copy: fda/FDA-Postmarket-Management-Cybersecurity-2016.pdf

S17 — IEC 81001-5-1:2021 — Interpretation Sheet 1

November 2025; software categories, risk transfer and security maintenance

Local copy: IEC+81001-5-1-2021+ISH1-2025.pdf

Local filenames identify copies in the sibling standards repository. Source publications remain there. Draft AI lifecycle guidance is background only. A process standard does not automatically require every internal record in a submission.